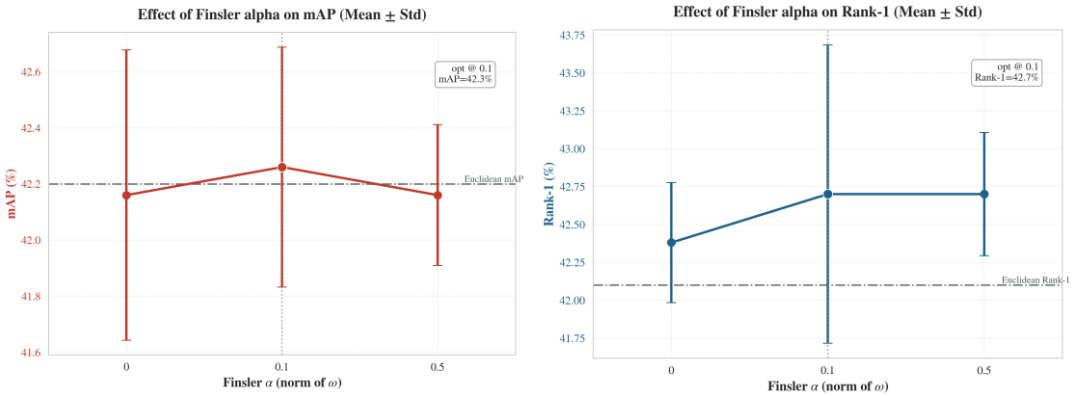
Finsler Alpha Sweep with Mean and Variance



Theory: Because TripletLoss relies on argmax to find the hardest negative, small numerical noise causes it to select different negatives, diverging the training trajectory significantly across runs (high variance). -> Ablate loss terms and see impact on variance

- Each alpha value ran 5 times in total, the values of mAP and Rank-1 retrieval averaged across all five runs to yield the mean
- The variances of both performance metrics are large -> might suggests instability in the model training
- Even different runs of the baseline (alpha = None) yields varying values of mAP and Rank-1
- Paper reports this value as 42.8 and 43.9?

- If there is a five runs you can plot them directly
- Stick to five is okay

- We have to see how baseline averaged

- What is the difference between rank-1 and mAP?
- Having alpha as a learnable hyperparameter might be useful here, learn perhaps a non-uniform alpha

Finsler Alpha Sweep with Mean and Variance

Alpha	mAP	mAP_mean	mAP_std	Rank-1	Rank-1_mean	Rank-1_std
None	42.2			42.1		
0	41.3			42.1		
0	42.6			42.4		
0	42.1	42.16	0.517687164	42.6	42.38	0.396232255
0	42.3			41.9		
0	42.5			42.9		
0.1	42.1			42.2		
0.1	42.1			42.4		
0.1	41.9	42.26	0.427784993	41.9	42.7	0.98488578
0.1	42.2			42.6		
0.1	43			44.4		
0.5	41.8			42.7		
0.5	42.4			42.9		
0.5	42.1	42.16	0.250998008	42.1	42.7	0.40620192
0.5	42.4			43.2		
0.5	42.1			42.6		

Re-running baseline gives a different value -> Perhaps we were just lucky this time?

Next: Copy a “clean” version of the BAU repository and run the exact same hyperparameter config 5 times; compute mean and variances

Same RNG experiment but in a tabular form

Variance / Discrepancy from Baseline not from Finsler term

```python
if alpha is not None:
    assert alpha < 1        # Sanity Check
    y_last = y[:, -1].unsqueeze(0)  # (1, n)
    x_last = x[:, -1].unsqueeze(1)  # (m, 1)
    finsler_term = (y_last - x_last) * alpha
    assert finsler_term.shape == dist.shape, "Finsler term shape mismatch"

    # DEBUG: Sanity check for alpha=0
    if alpha == 0:
        # If everything is correct, dist + finsler_term should be identical to dist, or at least
        # within a tiny float tolerarnce.

        # 1. Check for NaNs (e.g., if inputs had Inf, Inf * 0 = NaN)
        if torch.isnan(finsler_term).any():
            print(f"[DEBUG CRITICAL] alpha=0 but Finsler term contains NaNs! Check inputs for Infinity.")

        # 2. Check for values > 0 (should all be zeros)
        if not torch.allclose(finsler_term, torch.zeros_like(finsler_term)):
            max_val = finsler_term.abs().max().item()
            print(f"[DEBUG CRITICAL] alpha=0 but Finsler term is not zero! Max absolute value: {max_val}")

        # 3. Direct comparison of before/after distance matrices
        new_dist = dist + finsler_term
        diff = (dist - new_dist).abs()
        if diff.max() > 1e-7:  # Tolerance for float32 additions
            print(f"[DEBUG CRITICAL] alpha=0 distance matrix mismatch! Max diff: {diff.max().item()}")

        print(f"[DEBUG] alpha=0 sanity check completed.")
    dist = dist + finsler_term
```

log.txt

logs > finsler > job_1481113_alpha0.0 > log.txt

DEBUG CRITICAL No results

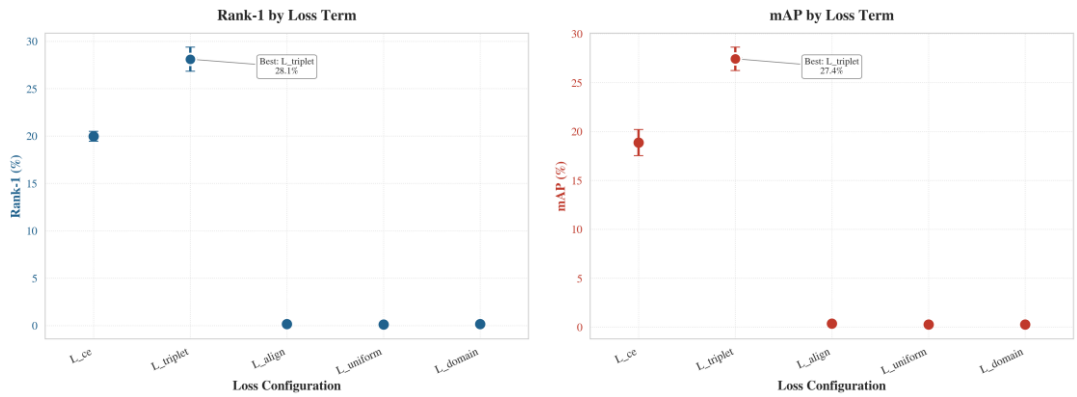
```
Trainer constructed
Starting training loop for 60 epochs
Epoch 1/60 - starting
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
[DEBUG] alpha=0 sanity check completed.
```

- Placed sanity check hooks in the computation of the finsler term, but only when alpha == 0
- Conducted three checks, none of them are called as evident in the log file on the right

- If we just change the last component, the meaning of alpha might be worthless if the vector is large
- Learning alpha adds a degree of freedom to the model, which introduces real dissimilarity in the model
- When we fix the alpha, we are rescaling the difference between the last components
- Beyond learning the alpha, we might learn omega (magnitude and direction)

- Add alpha as a parameter and learn it
- Use a pretrained mode and initialize with it (alpha=0)
- Add some constraints in computing alpha (apply a sigmoid to the scalar)

Individual Impact of Loss Terms on Variances in mAP and Rank-1 (alpha=0.0)



Alternative: Disable Triple Loss but keep the others

- Ablated all the loss terms, each run only has one single loss term running
- Confirms that alignment loss, uniform loss and domain loss have zero performance and zero variance when used in isolation (inert)
    - Regularization terms that require a feature structure
    - Not the source of instability

- Argmax instability supported by Rank-1 data
- But cross-entropy actually introduces more variance to the mAP score than triplet

- Keep something from the task when doing these experiments
- Keep cross entropy loss all the way?
- All the three regularization terms would work as

- Learning alpha needs to be at least as good (performance does not drop)

Individual Impact of Loss Terms on Variances in mAP and Rank-1 (alpha=0.0)

LossName	mAP	Rank-1	mAP_mean	mAP_std	Rank-1_mean	Rank-1_std
L_ce	17.4	19.4	18.86666667	1.331665624	19.96666667	0.513160144
L_ce	20	20.1				
L_ce	19.2	20.4				
L_triplet	27.8	28.4	27.43333333	1.193035345	28.1	1.276714533
L_triplet	28.4	29.2				
L_triplet	26.1	26.7				
L_align	0.3	0.1	0.366666667	0.115470054	0.166666667	0.115470054
L_align	0.3	0.1				
L_align	0.5	0.3				
L_uniform	0.2	0.1	0.266666667	0.057735027	0.1	0
L_uniform	0.3	0.1				
L_uniform	0.3	0.1				
L_domain	0.3	0.2	0.266666667	0.057735027	0.166666667	0.057735027
L_domain	0.2	0.1				
L_domain	0.3	0.2				

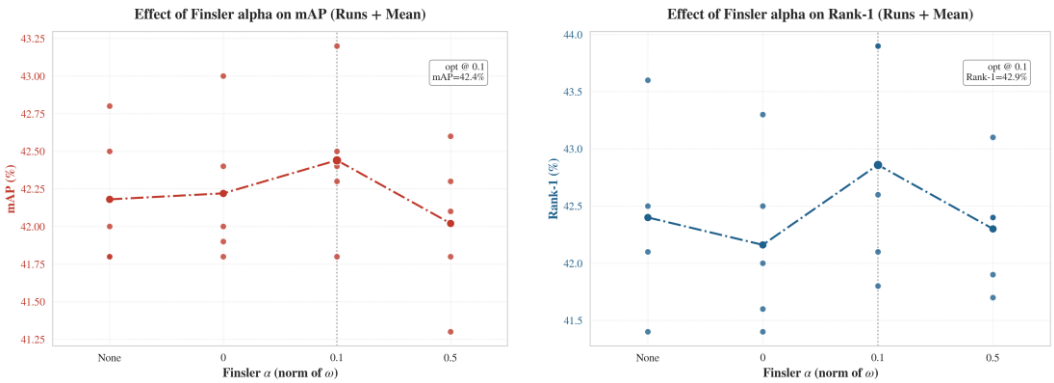
Today

1. Copy a “clean” version of the BAU repository and run the exact same hyperparameter config 5 times; compute mean and variances (seeing baseline std aligns with alpha=0)
   - Found a mistake in the evaluator -> rerun RNG alpha sweep for Finsler BAU
2. Find a configuration for learnable alpha where the performance is better or same
   - Higher chance for finsler to be better if the dataset suits finsler
   - Asymmetry between classes, persons, clothes
   - Draw the network that I used to learn alpha / formula to convey to Simon and Thomas on what I am doing
   - Initialize from scratch (alpha=0.0) and from pretrained models (BAU Baseline)

3. Implement learnable omega
4. Continue working on the visualisations

- Ran three main experiments: CE Loss, Triplet Loss and Alignment Loss

Finsler Alpha Sweep with Mean and Variance



- [mAP] Baseline and alpha=0 matches in terms of the mean value and variance
- [Rank-1] Still slight difference in mean and variance between baseline and sanity check

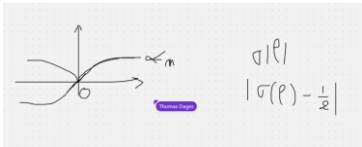
Both graphs point towards the initial conjecture that an alpha value of 0.1 appears optimal for this dataset configuration

- In the Euclidean case, there is a high variance
- In alpha=0.1, seem to concentrate around higher values
- Benefit is not that clear, difficult to see any contributions

- Not clear why we would hope for something better

Learnable Alpha – Model Introduction

Alpha Parameter



$0 < \alpha_{max} < 0$ be the maximum scaling factor (default 1.0).

$\rho \in R$ be the **learnable** logit parameter.

$$\alpha = \boxed{\alpha_{max}} \cdot \boxed{\sigma}\left(\frac{\boxed{\rho}}{\boxed{T}}\right)$$

$\sigma(z) = \frac{1}{1 + e^{-z}}$ be the sigmoid function.

$T > 0$ be the temperature scaling factor (default 1.0).

Why introduce the temperature hyperparameter?

- Control gradient sensitivity: Gradient of alpha w.r.t. raw pho is proportional to 1/T.
    - Since it is updated by the same global optimizer, the global step size might be too large/small for this single scalar.
    - Adjusting T acts as a local learning rate modifier
        - High T (> 1): Dampens the gradients
            - allows alpha to change slowly and smoothly, preventing it from oscillating wildly / saturating immediately
            - Smooths loss landscape w.r.t alpha, prevents optimization from being stuck in sharp local minima early in training
        - Low T (< 1): Amplified gradients -> forces alpha to converge quickly to a specific value
- Mitigating Vanishing Gradients (Sigmoid Saturations)
    - Sigmoid becomes very flat when absolute value of input (logits rho) is large
    - When initializing alpha to zero or close to alpha_max, rho must have a large magnitude

    - Gradient of the sigmoid is near zero -> freezes parameter
    - Increasing T shrinks the input back to zero
- Allows for annealing strategies (e.g., Gumbel-Softmax)

Alpha max parameter to limit the growth of alpha parameter

- Have to be careful with the activation function
- DL likes small numbers
- We want good behavior to be centered around zero (Euclidean), by large numbers could have very large deviation
- Challenging that the simplest behavior can scale in a stable way (because for raw alpha has to be large)
- Maybe using an exponential might be better (might be more stable) because sigmoid is not differentiable at zero

Learnable Alpha – Model Introduction

Canonical Randers Distance (swapping out Euclidean distance)

euclidean_distance(x, y, α)

Else: $d_{euc}((x, y)) = ||x - y||_2$

If alpha ≠ None: $d_{\alpha}((x, y)) = ||x - y||_2 + \alpha(y_D - x_D)$

Corresponds to a Randers metric where the drift vector ω is defined sparsely as $\omega = [0, 0, \dots, \alpha]^T$

1. Triplet Loss
2. Alignment Loss
3. Uniformity Loss
4. Domain-specific Uniformity Loss

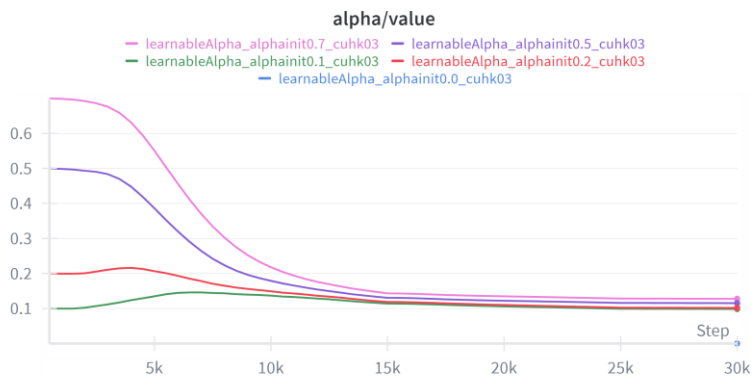
- X and y are two embedding vectors mapped by the network by two different inputs (dimension D)
- Adding alpha adds an asymmetric term based on the difference in the last dimension (D-th dimension)

- Might want to define different distance functions separately
    - We should have at the definition model the association to the distance functions

- Will be migrated into a finsler_distance in the future to accelerate computation on the GPU
    - Also cleaner code structure

Learnable Alpha – Model Introduction

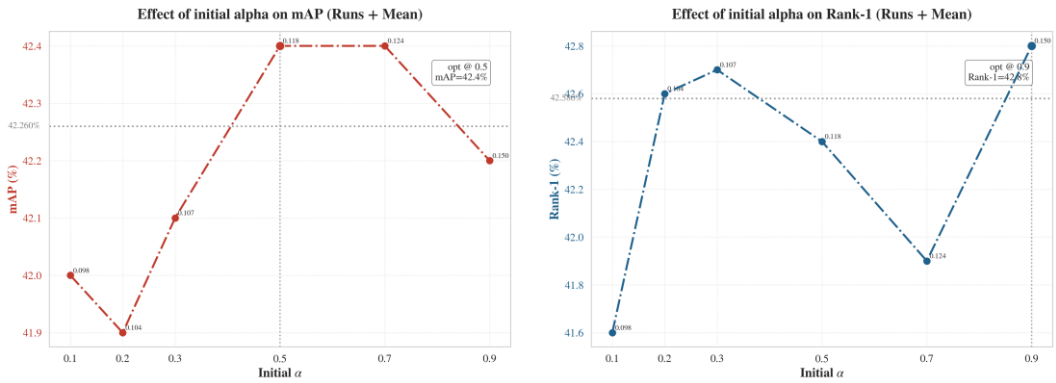
Results



- Also plot the initial rho values (several times to T)
- Rho values is exploding negatively
- Sigmoid curve is becoming flat

Learnable Alpha – Model Introduction

Results



- X and y are two embedding vectors mapped by the network by two different inputs (dimension D)
- Adding alpha adds an asymmetric term based on the difference in the last dimension (D-th dimension)

- This seems like a random seeding because they have the same final value but different mAP and Rank 1

- Will be migrated into a finsler_distance in the future to accelerate computation on the GPU
    - Also cleaner code structure

Shrink latent space to between 0 and 1
Learning a non-constant omega?

Next Steps

1. Think of other functions for alpha that does not cause the raw parameter to diverge
   - First show that it diverges, so that we can prove that it does not diverge with the new function
2. Try with loading a pretrained checkpoint on BAU
3. Double check the evaluator implementation
4. Loss ablation test similar to table 5 of the paper
   - Hypothesis: Not all losses should use the Finsler distance function or need special care
   - Start with this table and add different composite loss terms on-the-go
5. Brainstorm and implement architecture to learn drift vector omega
   - Right now the model can only model asymmetry if the "domain shift" happens to align perfectly with the last dimension
6. Continue working on the visualisations
   - We could plot pairs of images that have the highest magnitude of omega, and pairs with the smallest magnitude
   - Reduce the IDs in the query so that it is more manageable (perhaps only 10-15)
   - Find a way to keep these IDs always the same across runs (just for the visualization), and not dependent on how the dataset is shuffled given the seed

Ideas on learning the drift vector omega

- Modify the backbone to split into two branches after final pooling layer
  - Identity branch (standard embedding layer)
  - Drift branch: New MLP layer to predict the drift layer
    - Must have a normalization function to ensure that the magnitude of the vector is less than 1
    - Learning separate drift vector could lead to overfitting -> model just learns a drift that points directly to the ground truth gallery image
      - Enforce smoothness (e.g.,) or sparsity (e.g., use L1 regularization) on omega
      - L2 norm of one (strictly lower than 1)

Why I think alignment loss should be treated differently

Alignment loss minimizes distances between positive pairs (augmented and original image)

- If you augment an image x (source) to x´ (augmented/target), the distance should be measured with the drift of the source?

- In the canonical rander's distance, a positive w^T(y-x) increases distance
    - In DG, we might want the model to learn that "easy" augmentations (drift-aligned) have lower distances
    - Distances could be measured with the drift from the source

Drift consistency in augmentations

- BAU generates weakly and strongly augmented versions of the same image
- For both augmentations we could compute the features and drift vectors
- Hypothesis: Entropy increases from weak to strong domain
- We could constrain the difference in features to align with the omega vector of the weakly augmented image?
    - Transforms the "Drift Head" into a predictor of potential domain shifts -> essence of robust DG

Master Thesis?

Clean, simple and reproducible model

- Just do the cross entropy and align loss
- Good knowledge of the baseline
- See if some idea we have